Churn-Retention Optimization with Contextual Bandits

Fintech Hackathon Methodology Report

MARSILUX

April 18, 2026

1 Executive summary

This report presents a practical retention decision system for retail finance. The objective is to maximize expected net revenue increment from retention interventions, not only to minimize churn rate.

The method has two parts. Model 1 estimates 90-day churn risk from customer state. Model 2 is a contextual bandit that selects exactly one action for high-risk customers and learns continuously from delayed outcomes.

To avoid a common misconception, this is a one-step decision framework (contextual bandit), not a full sequential RL setup. The optimization target is immediate 90-day net benefit per intervention event.

2 Business objective and churn definition

Customer retention is a core strategic objective in retail finance because acquisition is costly and product relationships are long-term. In many institutions, risk scoring exists, but intervention assignment is still rule-based (for example, defaulting to manual calls), which is difficult to scale and often economically suboptimal.

The business problem is therefore twofold: identify who is likely to churn and decide what intervention should be assigned now. The proposed system addresses both under three practical constraints: heterogeneous customer behavior, non-negligible intervention costs, and delayed outcomes.

This report uses one global churn definition across the full portfolio:

A customer is considered churned if they have closed all of their products.

At decision time t_i, outcome is observed at $t_i + 90$ days. The label is

$$Y_i = \begin{cases} 1, & \text{if customer } i \text{ has closed all products by } t_i + 90, \\ 0, & \text{otherwise.} \end{cases}$$

This captures complete customer exit and is directly measurable.

3 Method overview

We use a two-step decision process repeated across the full customer base:

1. Estimate churn risk from current customer state.
2. For risk-above-threshold customers, assign one retention action that maximizes expected net benefit.

Outcomes are observed after 90 days, and model parameters are updated so that decision quality improves over time. This design replaces static, call-centered workflows with data-driven, cost-aware action selection.

4 Model details

4.1 Model 1: Churn-risk model (batch refreshed)

Let $i \in \{1, \ldots, n\}$ index decision events. The customer state is $x_i \in \mathbb{R}^d$, built from the most recent 9 months (demographics, transactions, days since last activity, digital engagement, product holdings, tenure, and balance summaries; optional segment/cluster/risk features).

The first model estimates churn probability on the 90-day horizon:

$$r(x_i) = \Pr(Y_i = 1 \mid X = x_i),$$

trained on historical supervised data

$$\mathcal{D}^{\text{risk}} = \{(x_i, Y_i)\}_{i=1}^{n}.$$

In the implemented notebook (`04_enhanced_features_model.ipynb`), Model 1 is trained as follows:

- **Reference timeline.** Features are computed on a fixed 9-month window from 2025-04-01 to 2025-12-31. At-risk population is defined at REF = 2026-01-01. Churn label is evaluated at CHURN_END = 2026-03-31.
- **Population and label construction.** The training cohort includes customers with *any active product* at REF. A customer is labeled churned if they had at least one active core product domain (ACCOUNTS/LOANS/DEPOSITS) at REF and had no active core products at CHURN_END.
- **Observed class balance.** $n = 7{,}805$ at-risk customers, with 131 churned (1.7%) and 7,674 active (98.3%).
- **Feature set.** All 20 features are engineered and derived from the dataset provided during the hackathon. The exact features used are: `tenure_years, n_products, has_loan, receives_salary, avg_txn_per_month, avg_txn_amount, txn_trend, avg_balance, balance_trend, n_contacts, age, credit_rating, has_deposit, digital_txn_ratio, debit_credit_ratio, balance_volatility, txn_amount_std, recency_days, n_complaint_contacts, avg_product_age_yea`
- **Missing-value handling.** Behavioral activity features are imputed with 0, `recency_days` is imputed with max window length, and demographic/relationship fields are median-imputed. Remaining missing values after cleaning: 0.
- **Model and tuning.** Classifier is LightGBM. Data is split using stratified 80/20 train-test split (random seed 42). SMOTE is applied on training folds only with sampling ratio 0.3. Hyperparameters are tuned with Optuna (150 trials, 5-fold stratified CV), maximizing best-in-fold F1 over threshold sweep.
- **Thresholding and output.** Predicted probabilities are thresholded by maximizing test-set F1, giving threshold $t = 0.88$ and F1 = 0.5833 (precision 0.6364, recall 0.5385), versus notebook-03 baseline F1 = 0.3860.

Model 1 outputs calibrated/normalized risk scores `churn_risk_score`$\in [0, 1]$, exported for downstream activation of the action policy.

4.2 Model 2: Retention action model (online contextual bandit)

The second model is a contextual bandit used after risk-based activation for one-shot retention assignment over

$$\mathcal{A} = \{a_0, \ldots, a_{K-1}\},$$

where exactly one action $A_i \in \mathcal{A}$ is assigned per customer event. Action learning is performed only on activated events satisfying $r(x_i) > t$.

To ensure robust learning in a hackathon setting, we use a compact and practical action set:

$$\mathcal{A} = \{\text{no action},\ \text{push notification},\ \text{email},\ \text{call}\}.$$

Table 1: Retention action set with illustrative normalized costs

Action $a \in \mathcal{A}$	Cost $c(a)$
a_0: No action	0.00
a_1: Push notification	0.05
a_2: Email	0.10
a_3: Call	0.90

Let $c(a) \geq 0$ denote action cost and $\lambda > 0$ a cost weight. The realized reward is

$$R_i = (1 - Y_i) - \lambda c(A_i),$$

or, when customer value differs substantially,

$$R_i = (1 - Y_i)v_i - \lambda c(A_i),$$

where v_i is an estimate of customer value.

For each action a, the learning target is

$$\mu_a(x) = \mathbb{E}[R \mid X = x, A = a], \qquad \pi^\star(x) \in \arg\max_{a \in \mathcal{A}} \mu_a(x).$$

Thus, for each customer profile, the system selects the retention measure with the highest expected net benefit.

Online parameter update. For implementation, each action has its own parametric reward model

$$\widehat{\mu}_a(x) = \theta_a^\top x,$$

with parameters $\theta_a \in \mathbb{R}^d$. After assigning action A_i to context x_i and observing delayed reward R_i, update only the parameters of the chosen action:

$$\theta_{A_i} \leftarrow \theta_{A_i} + \eta\big(R_i - \theta_{A_i}^\top x_i\big)x_i,$$

where $\eta > 0$ is the learning rate. In practice, features should be normalized and a small regularization term can be added for numerical stability. Parameters for actions not chosen at event i remain unchanged.

5 Operational pipeline

The customer journey through the system has five stages:

0) Train churn-risk model on past data. Build

$$\mathcal{D}^{\text{risk}} = \{(x_i, Y_i)\}_{i=1}^n,$$

fit $r(x_i) \approx \Pr(Y_i = 1 \mid X = x_i)$ using the enhanced 20-feature table and update this model periodically in batch mode as new (x_i, Y_i) labels become available.

1) Activation gate. For current customers, compute $r(x_i)$. Enter retention decisioning only if

$$r(x_i) > t.$$

This focuses spend on customers with meaningful churn risk.

2) Action assignment. For activated customers, estimate expected reward per action,

$$\widehat{\mu}_a(x_i) \approx \mathbb{E}[R \mid X = x_i, A = a],$$

and assign

$$A_i = \arg\max_{a \in \mathcal{A}} \widehat{\mu}_a(x_i),$$

with limited exploration (for example, ε-greedy) for ongoing learning and identifiability of action effects.

3) Observe 90-day results. Check whether the customer has closed all products. Compute Y_i and realized reward R_i.

4) Update models. Append new observations as $(x_i, a_i, v_i, r_i, p_i)$, where v_i is `customer_value_score` and $p_i = \Pr(A_i = a_i \mid x_i)$ is the logged action propensity under the behavior policy. For each new event, update only the chosen action model using

$$\theta_{a_i} \leftarrow \theta_{a_i} + \eta\big(r_i - \theta_{a_i}^\top x_i\big)x_i,$$

while parameters of non-chosen actions remain unchanged. Refresh the churn-risk model periodically in batch mode with newly available (x_i, Y_i) labels.

6 Data requirements and evaluation protocol

Offline action learning requires logged intervention data:

$$\mathcal{D}^{\text{bandit}} = \{(x_i, a_i, v_i, r_i, p_i)\}_{i=1}^n.$$

For targeted campaigns, a filtered view is

$$\mathcal{D}_t = \{(x_i, a_i, v_i, r_i, p_i) \mid r(x_i) > t\}_{i=1}^n.$$

For hackathon scope, the minimal fields beyond context are action a_i, customer value score v_i (stored as `customer_value_score`), realized reward r_i at 90 days, and logged propensity p_i. The tuple (a_i, v_i, r_i) is sufficient for online updates, while p_i is needed for principled offline policy evaluation.

Table 2: Logged dataset schema for contextual-bandit training

Field	Type	Description
x_i	Vector	Customer state at decision time, including demographics, engagement, product holdings, and 9 months of behavioral history.
a_i	Categorical	Historical retention action assigned to customer i.
v_i	Real	`customer_value_score` used for value-aware optimization.
r_i	Real	Observed reward at $t_i + 90$ days.
p_i	Real	Logged action propensity $\Pr(A_i = a_i \mid x_i)$ under the behavior policy.

7 Proof of concept: does Model 2 learn?

To verify that the contextual-bandit component can learn a meaningful policy, we run a controlled proof of concept with a synthetic training log for Model 2.

Synthetic data design. We generate records of the form $(x_i, a_i, v_i, r_i, p_i)$ with the same schema used in deployment. The generative behavior is intentionally asymmetric:

- Actions a_0 (no action), a_1 (push), and a_2 (email) have low statistical retention effect.
- Action a_3 (call) has high statistical retention effect.

Formally, for churn label simulation we set

$$\Pr(Y_i = 0 \mid A_i = a_3, x_i) \gg \Pr(Y_i = 0 \mid A_i \in \{a_0, a_1, a_2\}, x_i).$$

At the same time, a_3 carries the highest cost $c(a_3)$, so calls are not always optimal under a revenue-aware reward.

Expected learning behavior. Under

$$R_i = (1 - Y_i)v_i - \lambda c(A_i),$$

the learned policy should favor a_3 when expected retention value is high, but avoid a_3 for low-value or low-uplift cases where call cost dominates. This is the key test: the model should not collapse to "always call."

Validation diagnostics on holdout data.

1. Compare empirical churn outcomes and average reward by action on the test set.
2. Report model-selected action mix by customer value deciles (based on v_i).
3. Check that call selection probability increases with expected value/uplift, not uniformly for all customers.
4. Compare total expected reward of the learned policy against simple baselines (always a_0, always a_3, and uniform random action), using an offline estimator that accounts for propensities.

Success criterion: the learned policy allocates many calls where they are economically justified, but still withholds calls in segments where they would generate negative revenue increment. A secondary check is parameter dynamics: the estimated call-model coefficients θ_{a_3} should stabilize as evidence accumulates, while lower-effect actions show weaker reward-aligned updates.

8 Practical limitations and scope

This approach is appropriate for a hackathon because it is mathematically rigorous, aligned with a two-step workflow (risk gating plus one-shot intervention), and explainable to business and technical reviewers.

Main limitations are delayed reward, dependence on logged action data quality, possible selection bias induced by risk gating, and no explicit modeling of long multi-step customer journeys.

9 Conclusion

We combine churn awareness and action optimization in one coherent system for the full customer population. The risk model identifies likely churn in the next 90 days from a 9-month state, and the contextual bandit assigns one retention action to maximize expected net benefit.

The primary contribution is decision quality: not only estimating who may churn, but selecting which retention measure should be assigned now, then learning from observed 90-day outcomes.